

1부: 기초 AI (Part 1: Basic AI)

이 파트에서는 게임 AI의 목표를 알아보고, 게임 내에서 간단하지만 효과적인 AI 캐릭터를 구현(implement)하기 위한 기본 기법(techniques)을 학습합니다.

이 파트에서 다루는 장은 다음과 같습니다:

- 1장, 'AI 소개 (Introduction to AI)'
- 2장, '유한 상태 기계 (Finite State Machines)'
- 3장, '무작위성과 확률 (Randomness and Probability)'
- 4장, '센서 구현 (Implementing Sensors)'

제1장: 인공지능 소개

이 책은 가장 인기 있는 상용 게임 엔진 중 하나인 Unity3D를 사용하여 비디오 게임용 인공지능 (AI) 프로그래밍 의 기초를 가르치는 것을 목표로 합니다 . 앞으로 이어지는 장에서는 행동 트리와 유한 상태 머신과 같은 현대 게임의 기본 기술들을 구현하는 방법을 배우게 될 것입니다.

그 전에, 먼저 인공지능(AI)에 대한 기본적인 학문적 배경지식을 갖추어야 합니다. 이 소개 장에서 그 내용을 다룰 것입니다. 다음으로, 게임 분야에서의 AI 적용 및 구현 방식이 다른 분야와 어떻게 다른지, 그리고 게임 AI에 필수적인 고유한 요구 사항은 무엇인지 알아보겠습니다. 마지막으로, 게임에 사용되는 기본적인 AI 기술들을 살펴보겠습니다.

이 장에서는 다음 주제들을 다룰 것입니다.

- 인공지능 이해하기
- 비디오 게임 속 인공지능
- 비디오 게임을 위한 AI 기술

인공지능 이해하기

지능(Intelligence)은 동물과 인간 같은 모든 생명체에게 자연스럽고 필수적인 요소입니다. 여기서 가능한 가장 넓은 의미로 지능을 언급하자면, 지능이 없다면 동물은 먹이를 찾지 못할 것이고, 꿀벌은 꽃을 찾지 못할 것이며, 우리 인간은 유니티로 게임을 개발하는 것은 고사하고 물건을 만들거나 불을 피우지도 못했을 것입니다. 반대로, 컴퓨터는 데이터를 입력받아 고속으로 논리 및 수학적 연산을 수행하고 결과를 출력할 수 있는 전자 기기일 뿐입니다. 컴퓨터에는 어떠한 지능도 없습니다. 누군가가 무엇을 어떻게 해야 할지 지시하지 않는다면, 컴퓨터는 돌처럼 영원히 움직임 없이 생명력 없는 상태로 머물러 있을 것입니다.

이러한 관점에서 인공지능은 본질적으로 기계에 자연 지능의 불꽃을 불어넣는 방법을 연구하는 분야입니다. 인간의 개입 없이 어떤 목표든 달성하기 위해 컴퓨터가 살아있는 유기체처럼 생각하고 결정하는

방법을 가르치는 학문입니다.

짐작하시겠지만, 인공지능은 방대한 주제입니다. 이렇게 작은 책 한 권으로 인공지능과 관련된 모든 것을 다룰 수는 없습니다. 다행히 게임 인공지능이라는 목표를 위해서는 인공지능에 대한 포괄적인 지식이 필요하지 않습니다. 기본적인 개념을 이해하고 기초적인 방법만 익히면 됩니다. 기술들입니다. 그리고 이것이 바로 이 책에서 우리가 다룰 내용입니다..

하지만 게임별 기술로 넘어가기 전에 인공지능의 주요 연구 분야 몇 가지를 살펴보겠습니다.

- **컴퓨터 비전(Computer vision):** 비디오나 사진과 같은 시각적 소스에서 시각적 입력을 받아 이를 분석하여 객체를 식별(객체 인식, object recognition)하거나, 얼굴을 식별(얼굴 인식, face recognition)하거나, 손글씨 문서의 텍스트를 식별(광학 문자 인식, optical character recognition)하고, 심지어 입체 이미지에서 3D 모델을 재구성하는 능력입니다.
- **자연어 처리(Natural Language Processing, NLP):** 기계가 인간의 언어, 즉 우리가 쓰고 말하는 방식을 읽고 이해할 수 있게 해줍니다. 문제는 인간의 언어가 기계가 이해하기 어렵다는 것입니다. 언어의 모호성(ambiguity)이 주된 문제인데, 같은 것을 말하는 데는 여러 가지 방법이 있으며, 같은 문장이라도 문맥에 따라 다른 의미를 가질 수 있습니다. NLP는 기계가 언어와 표현을 처리하고 그에 맞게 반응하기 전에 이를 이해해야 하므로, 기계에 있어 중요한 인지적 발전 단계입니다. 다행히 연구자들이 이 복잡한 작업을 위해 기계를 훈련하는 데 도움이 되는 많은 데이터셋(datasets)이 웹에 공개되어 있습니다.
- **머신 러닝(Machine learning, 기계학습):** : 인공지능의 한 분야인 머신 러닝은 인간의 개입 여부와 관계없이, 오직 원시 데이터와 경험만을 이용하여 기계가 작업을 수행하는 방법을 학습하는 방식을 연구합니다. 이러한 작업은 사진에서 고양이 이미지를 식별하는 것부터, 보드게임 플레이(2017년 바둑에서 세계 1위 플레이어를 이긴 알파고(AlphaGo) 소프트웨어처럼), 우리가 만든 홈 비디오에 유명 배우의 얼굴을 완벽하게 합성하는 것(소위 딥페이크, deepfakes)까지 다양합니다. 머신러닝은 다른 모든 AI 분야를 아우르는 방대한 분야입니다. 이에 대해서는 11장, '유니티에서의 머신러닝'에서 자세히 다룰 것입니다.
- **상식적 추론 (Common sense reasoning):** 인간에게는 거의 선천적으로 타고난 지식의 한 유형이 있습니다. 예를 들어, *물건이 지지되지 않으면 땅에 떨어진다는 것*, 큰 물건을 작은 물건 안에 넣을 수 없다는 것은 너무나 당연한 사실입니다. 그러나 이러한 종류의 지식과 추론(상식 지식, common sense knowledge이라고도 함)은 컴퓨터에게는 완전히 해독 불가능한 것입니다. 이 글을 쓰는 시점까지 기계에게 우리에게 너무나 당연한 이런 것들을 가르치는 방법을 아는 사람은 아무도 없습니다. 그럼에도 불구하고, 이는 매우 활발히(그리고 좌절감을 주며) 연구되는 방향입니다.

다행스럽게도, 게임 AI의 범위는 훨씬 좁습니다. 대신 다음 섹션에서 보게 되겠지만, 게임 AI는 단 하나의 필수적인 목표를 가집니다. 바로 게임을 재미있게 만드는 것입니다.

비디오 게임 속 인공지능

일반적인 AI와 달리, 게임 AI는 단지 '지능의 환상(illusion of intelligence)'만을 제공하면 됩니다. 게임 AI의 목표는 인간과 같은 지능형 에이전트(intelligent agents)를 제공하는 것이 아니라, 게임을 재미있게 만들 수 있을 만큼만 똑똑한 캐릭터를 제공하는 것입니다.

물론, 게임을 재미있게 만드는 것은 결코 간단한 문제가 아니며, 공정하게 말하자면 훌륭한 AI는 이 문제의 일부일 뿐입니다. 그럼에도 불구하고 훌륭한 AI가 게임을 재미있게 만들기에 충분하지 않다 하더라도, 나쁜 AI는 아무리 잘 설계된 게임이라도 망칠 수 있습니다. 무엇이 게임을 재미있게 만드는지에 대한 문제에 관심이 있다면, 제시 셸(Jesse Schell)이 쓴 *게임 디자인의 예술(The Art of Game Design)*과 같은 훌륭한 게임 디자인 책을 읽어보는 것을 추천합니다.

하지만 우리의 관심사로 한정 지어 말하자면, 플레이어에게 적절한 수준의 도전을 제공하는 것이 필수적이라고 말할 수 있습니다. 여기서 공정한 도전이란, 게임이 너무 어려워서 플레이어가 상대를 이길 수 없거나, 반대로 너무 쉬워서 이기는 것이 지루한 작업이 되어서는 안 된다는 것을 의미합니다. 따라서 적절한 도전 수준을 찾는 것이 게임을 재미있게 만드는 핵심입니다.

그리고 바로 여기서 AI가 개입합니다. 게임에서 AI의 역할은 게임 세계에서 적절하게 행동하는 도전적인 상대와 흥미로운 논플레이어 캐릭터(NPC, Non-Player Character)를 제공하여 게임을 재미있게 만드는 것입니다. 따라서 여기서의 목표는 인간이나 동물의 전체 사고 과정을 복제하는 것이 아니라, 게임 세계의 변화하는 상황에 반응하여 플레이어에게 납득할 수 있는 방식으로 행동함으로써 NPC가 똑똑해 보이도록 만드는 것입니다. 앞서 언급했듯이 이것이 바로 지능의 환상을 제공하는 것입니다.

정보

게임 내 AI가 NPC의 행동을 모델링하는 것에만 국한되지 않는다는 점을 언급하는 것은 중요합니다. AI는 게임 콘텐츠를 생성하거나(10장 '절차적 콘텐츠 생성'에서 살펴볼 예정입니다), 스토리 이벤트와 서사 속도(narrative pace)를 제어하거나(레프트 포 데드(Left 4 Dead) 시리즈의 AI 디렉터가 대표적인 예입니다), 심지어 전체 서사 아크(narrative arcs)를 만들어 내는 데에도 사용됩니다.

훌륭한 게임 AI가 반드시 복잡한 AI일 필요는 없다는 점에 유의하세요. 자주 언급되는 예로 오리지널 아케이드 게임 팩맨(Pac-Man)의 AI가 있습니다. 최신 기준으로 볼 때 팩맨을 쫓는 네 마리 유형의 행동을 제어하는 알고리즘은 AI라고 부르기조차 민망할 정도입니다. 각 유형은 다음에 어디로 이동할지 결정하기 위해 아주 단순한 규칙을 사용합니다. 유형과 타겟 타일(target tile) 사이의 거리를 측정하고 그 거리를 최소화하는 방향을 선택하는 것입니다.

이 타겟 타일은 팩맨 자신의 위치일 수도 있지만(빨간 유형의 경우), 팩맨의 앞쪽 공간이거나(분홍 유형의 경우) 다른 어떤 타일일 수도 있습니다. 단순히 타겟 타일의 위치를 변경하는 것만으로, 아케이드 게임 팩맨은 각 유형에게 독특한 성격을 부여하고 40년이 지난 지금도 우리에게 도전을 안겨주는 AI를 만들어낼 수 있었습니다!

여기서 황금률(golden rule)은 게임 디자인의 목표를 달성하는 데 필요한 최소한의 AI만 사용하는 것입니다. 물론 우리는 이 규칙을 극단으로 밀어붙여 AI가 불필요하다고 판단되면 AI를 전혀 사용하지 않을 수도 있습니다. 예를 들어, 포탈(Portal)과 포탈 2(Portal 2)에서는 모든 캐릭터가 완전히 스크립트로 짜여져 있으며(scripted) AI가 전혀 개입하지 않지만, 아무도 AI가 없다고 불평하지 않았습니다.

정보

팩맨 AI에 대해 더 자세히 알아보고 싶다면, Retro Game Mechanics Explained 유튜브 채널의 아주 상세한 영상을 시청하시길 추천합니다: <https://www.youtube.com/watch?v=ataGotQ7ir8>

혹은 읽는 것을 선호한다면 다음의 매우 유익한 웹페이지를 방문해 보세요

요: <https://gameinternals.com/understanding-pac-man-ghost-behavior>

게임 AI의 또 다른 과제는 그래픽 렌더링(graphics rendering) 및 물리 시뮬레이션(physics simulation)과 같은 다른 작업들이 AI 연산에 필요한 처리 능력(processing power)을 공유해야 한다는 점입니다. 그리고 이 모든 것이 실시간(real time)으로 일어나고 있다는 것을 잊지 마세요. 따라서 게임 내내 안정적인 프레임 속도(frame rate)를 유지하는 것이 매우 중요합니다. 이는 게임 AI가 컴퓨팅 리소스를 독차지하지 않도록 설계되어야 함을 의미합니다. 이는 일반적으로 알고리즘이 중간에 중단될 수 있고 여러 프레임에 걸쳐 분산 처리될 수 있도록 설계함으로써 이루어집니다.

일반 AI 분야에서는 많은 기업이 AI 연산을 위해 AI 가속기(AI accelerator, 예: 구글의 텐서 처리 장치 TPU)라는 전용 프로세서에 투자하고 있습니다. 그러나 게임이 이러한 전용 AI 프로세서를 광범위하게 사용할 수 있게 되기 전까지, 우리 게임 AI 개발자들은 여전히 알고리즘의 성능(performance)에 주의를 기울여야 합니다.

다음 섹션에서는 비디오 게임에서 가장 널리 사용되는 AI 기법들에 대한 전반적인 소개를 제공할 것입니다.

비디오 게임을 위한 AI 기술

이 섹션에서는 다양한 유형의 게임에서 흔히 사용되는 AI 기법들을 살펴보겠습니다. 앞으로의 장들에서 유니티에 이러한 기능들을 구현하는 방법을 배울 것입니다. 이 책은 AI 기법 그 자체보다는 유니티 내부에서 이 기법들을 구현하는 데 중점을 두고 있으므로, 여기서는 너무 자세히 다루지는 않겠습니다. 그러니 세부 구현으로 뛰어들기 전 속성 코스(crash course) 정도로 여겨 주시기 바랍니다.

게임 AI에 대해 더 깊이 배우고 싶다면 매트 버클랜드(Mat Buckland)의 Programming Game AI by Example, 이언 밀링턴(Ian Millington)과 존 펀지(John Funge)의 Artificial Intelligence for Games와 같은 훌륭한 책들이 있습니다. 또한, AI Game Programming Wisdom 및 Game AI Pro 시리즈도 최신 AI 기법에 대한 많은 귀중한 자료와 기사들을 포함하고 있습니다.

유한 상태 기계

유한 상태 기계(Finite State Machines, FSM)는 아마도 가장 단순하고, 가장 널리 사용되며, 가장 많이 논의되는 AI 모델 중 하나일 것이며, 대다수 게임에서 유일한 AI 기법으로 사용됩니다. 상태 기계는 하나 이상의 전이(transitions)로 연결된 유한한 개수의 상태(states)로 구성되며, 이는 그래프(graph) 형태의 자료 구조가 됩니다. 각 게임 엔티티(entity)는 초기 상태(initial state)에서 시작합니다. 그런 다음 환경 이벤트가 특정 규칙을 트리거하여 엔티티를 다른 상태로 이동시킵니다. 이러한 트리거 규칙을 전이(transitions)라고 부릅니다. 게임 엔티티는 특정 시점에 오직 하나의 상태에만 존재할 수 있습니다.

예를 들어, 전형적인 슈팅 게임의 AI 경비병 캐릭터를 생각해 보겠습니다. 그 상태는 정찰(patrolling), 추적(chasing), 사격(shooting)처럼 단순할 수 있습니다.

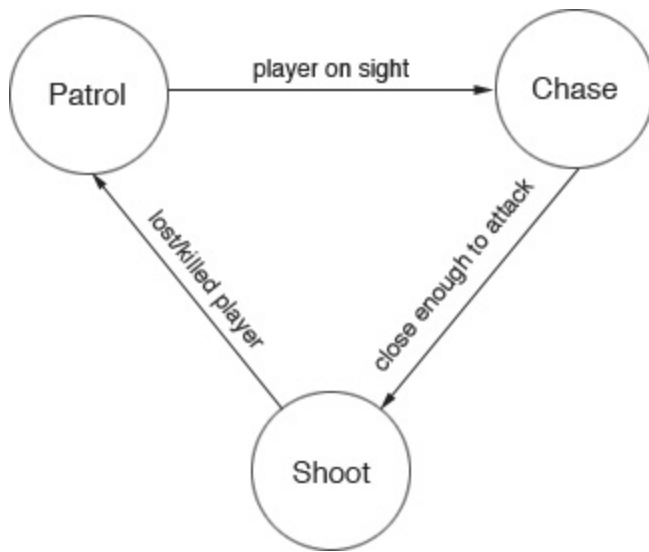


그림 1.1 – AI 경비 캐릭터를 위한 간단한 유한 상태 기계(FSM)

간단한 유한 상태 기계(FSM)에는 네 가지 구성 요소가 있습니다.

- **상태(States):** 게임 엔티티나 NPC가 선택할 수 있는 상태들의 집합을 정의합니다(예: 정찰, 추적, 사격).
- **전이(Transitions):** 서로 다른 상태 간의 관계를 정의합니다.
- **규칙(Rules):** 상태 전이를 언제 수행할지 정의합니다(예: 플레이어 발견, 공격 사거리 내 진입, 플레이어 시야에서 놓침/사살).
- **이벤트(Events):** 규칙을 확인할 트리거 역할을 하는 구성 요소입니다(예: 경비병의 시야 영역, 플레이어와의 거리 등).

따라서 *퀘이크 2*의 몬스터는 서 있기, 걷기, 달리기, 피하기, 공격하기, 대기하기, 탐색하기와 같은 상태를 가질 수 있습니다.

FSM은 유한 상태 머신(FSM)은 *if* 또는 *switch* 문 몇 개만으로 간단하게 구현할 수 있어 게임에서 널리 사용되지만, 간단한 게임부터 다소 복잡한 게임까지 충분히 강력한 기능을 제공합니다. 하지만 상태와 전환이 많아지면 코드가 복잡해질 수 있습니다. 이번 글에서는 간단한 FSM을 관리하는 방법을 알아보겠습니다. 다음 장에서.

인공지능에서의 무작위성과 확률(Randomness and probability in AI)

1인칭 슈팅(FPS) 게임에서 항상 헤드샷으로 플레이어를 죽이는 적 봇(bot)이나, 레이싱 게임에서 항상 최적의 루트만 선택하고 장애물에 절대 부딪히지 않는 상대를 상상해 보십시오. 이러한 수준의 지능은 게임을 너무 어렵게 만들어 승리하는 것을 거의 불가능하게 만들며, 결과적으로 플레이어에게 좌절감을 줄 것입니다. 반대의 스펙트럼에서, 플레이어로부터 도망칠 때마다 항상 똑같은 예측 가능한 경로를 선택하는 적을 상상해 보십시오. 몇 번의 게임 후, 플레이어는 적의 패턴을 파악하게 되고 게임은 지루해질 것입니다. 플레이어가 마주칠 때마다 항상 똑같은 방식으로 행동하는 AI 제어 엔티티는 게임을 예측 가능하고, 쉽게 이길 수 있게 하며, 따라서 따분하게 만듭니다.

물론 의도적인 예측 가능성이 요구되는 경우도 있습니다. 예를 들어 스텔스 게임(stealth games)에서는 플레이어가 몰래 이동할 경로를 계획할 수 있도록 적의 경로를 예측할 수 있게 만들어야 합니다. 하지만 다른 경우에는, 의도치 않은 예측 가능성이 게임의 몰입도를 방해하고 플레이어로 하여금 게임이 충분히 도전적이지 않거나 공정하지 않다고 느끼게 할 수 있습니다. 이러한 너무 완벽하거나 너무 멍청한 AI를 수정하는 한 가지 방법은 그들의 행동에 의도적인 실수(intentional mistakes)를 도입하는 것입니다. 게임에서 우리는 AI 계산의 의사 결정 과정에 무작위성(randomness)과 확률(probability)을 도입합니다.

약간의 무작위성을 도입하고 싶어지는 시나리오는 다양합니다. 가장 간단한 경우는 NPC가 아무런 정보가 없거나 혹은 어떤 결정을 내리든 상관없는 상황입니다. 예를 들어, 슈팅 게임에서 공격을 받는 적이 어디로 숨을지 결정해야 한다고 합시다. 이때 NPC에게 항상 가장 가까운 장애물로 이동하도록 하는 대신, 가끔은 조금 멀리 떨어진 장애물을 선택하도록 지시할 수 있습니다.

다른 경우에는 결정의 결과에 무작위성을 사용할 수 있습니다. 예를 들어 명중 확률에 무작위성을 적용하거나, 기본 대미지에 무작위로 약간의 피해량을 더하거나 빼거나, NPC가 사격을 시작하기 전에 주저하게 만들 수 있습니다.

센서 시스템(The sensor system)

우리의 AI 캐릭터는 특정한 결정을 내리기 위해 주변 환경과 상호작용하는 세계에 대해 알아야 합니다. 이러한 정보에는 다음이 포함됩니다.

- **플레이어의 위치:** 공격할지, 추적할지, 아니면 계속 정찰할지를 결정하는 데 사용됩니다.
- **건물 및 근처 오브젝트:** 숨거나 장애하는 데 사용됩니다.
- **플레이어의 체력 및 AI의 체력:** 후퇴할지 전진할지를 결정하는 데 사용됩니다.
- **실시간 전략(RTS) 게임의 맵에 있는 자원의 위치:** 다른 유닛을 업그레이드하거나 생산하는 데 필요한 자원을 점령하고 수집하는 데 사용됩니다.

상상할 수 있듯이 게임 정보를 수집하는 올바른 방법을 선택하는 것은 우리가 만들고자 하는 게임의 유형에 따라 크게 달라질 수 있습니다. 다음 두 섹션에서는 폴링(polling)과 메시지(이벤트) 시스템(messaging systems)이라는 두 가지 기본 전략을 살펴봅니다.

폴링 (Polling)

이러한 정보를 수집하는 한 가지 방법은 폴링(Polling)입니다. 폴링은 AI 캐릭터의 유니티 FixedUpdate 메서드에서 직접 앞서 언급한 정보들을 확인하는 방식으로 구성됩니다. 이렇게 하면 AI 캐릭터는 관심 있는 정보를 게임 월드에서 주기적으로 폴링(조회)하고, 확인 작업을 거친 뒤 그에 따라 행동을 취할 수 있습니다. 확인해야 할 항목이 너무 많지 않다면 폴링은 아주 잘 작동합니다.

이 방법을 더 효율적으로 만들려면, 모든 캐릭터가 모든 것을 한 번에 확인하지 않도록 월드 상태(world states)를 각기 다른 주기로 폴링하게끔 캐릭터를 프로그래밍할 수 있습니다. 예를 들어 폴링 에이전트를 10개 그룹(G1, G2, G3 등)으로 나누고, 각 그룹의 폴링 프레임을 분산 할당할 수 있습니다(예: G1은 0, 60, 120프레임에서 폴링하고, G2는 10, 70, 130프레임에서 폴링하는 식).

또 다른 예로, 적의 유형이나 상태에 따라 폴링 빈도(polling frequency)를 변경할 수 있습니다. 예를 들어, 전투 상태가 아니고 멀리 떨어져 있는 적은 3~4초마다 폴링하고, 플레이어와 가까이 있으며 공격을 받는 적은 0.5초마다 폴링하도록 할 수 있습니다.

하지만 게임 규모가 커지면 폴링만으로는 더 이상 충분하지 않습니다. 따라서 더 복잡한 AI 시스템을 갖춘 대규모 게임에서는 글로벌 메시징 시스템(global messaging system)을 사용하여 이벤트 기반 방법(event-driven method)을 구현해야 합니다.

메시징 시스템(Messaging systems)

메시징 시스템에서는 AI 엔티티와 플레이어, 월드, 또는 다른 AI 엔티티 간의 이벤트를 비동기 메시지(asynchronous messages)를 통해 통신합니다. 예를 들어 플레이어가 정찰병 그룹 내의 한 적 유닛을 공격하면, 다른 AI 유닛들은 이 사건에 대해 알아야만 플레이어를 찾고 공격을 시작할 수 있습니다.

만약 폴링 방법을 사용했다면, 우리의 AI 엔티티들은 동료 중 누군가가 공격받았는지 알아내기 위해 다른 모든 AI 엔티티의 상태를 계속 확인해야 했을 것입니다. 하지만 이 기능을 더 관리하기 쉽고 확장 가능한(scalable) 방식으로 구현할 수 있습니다. 특정 이벤트에 관심이 있는 AI 캐릭터들을 해당 이벤트의 리스너(listeners)로 등록하는 것입니다. 그러면 이벤트가 발생했을 때 메시징 시스템이 모든 리스너에게 이 정보를 브로드캐스트(broadcast, 전파)합니다. AI 엔티티는 그에 따른 적절한 조치를 취하거나 추가적인 확인을 수행할 수 있습니다.

이 이벤트 기반 시스템이 폴링보다 무조건 더 빠른 메커니즘을 제공하는 것은 아닙니다. 하지만 각 에이전트가 매 프레임마다 동일한 이벤트를 검사하게 하는 대신, 월드의 상태를 감지하고 관심 있는 AI 에이전트에게 알려주는 편리한 중앙 집중식 검사 시스템을 제공합니다. 실제로는 폴링과 메시징 시스템이 함께 사용되는 경우가 많습니다. 예를 들어, AI는 메시징 시스템으로부터 이벤트를 수신했을 때 더 자세한 정보를 얻기 위해 폴링을 수행할 수 있습니다.

플로킹, 스워밍, 허딩 (군집 행동)

새, 물고기, 곤충, 육상 동물 등 많은 생명체는 이동, 사냥, 먹이 찾기 등의 특정 활동을 무리 지어 수행합니다. 이들이 무리(groups) 내에 머물며 사냥하는 이유는 개별적으로 목표를 추구할 때보다 더 강해지고 포식자로부터 더 안전해지기 때문입니다. 그렇다면 하늘에서 새 떼가 무리 지어(flocking) 비행하거나 떼 지어 날아다니는(swarming) 모습을 만들고 싶다고 해봅시다. 애니메이터가 각 새의 움직임과 애니메이션을 일일이 디자인하려면 너무 많은 시간과 노력이 들 것입니다. 하지만 각 새가 따를 몇 가지 간단한 규칙을 적용하면, 무리 전체가 복잡하고 전역적인 행동을 하는 창발적 지능(emergent intelligence)을 달성할 수 있습니다.

이 개념의 선구자 중 한 명은 크레이그 레이놀즈(Craig Reynolds)로, 그는 1987년 시그그래프(SIGGRAPH) 논문 Flocks, Herds, and Schools – A Distributed Behavioral Model에서 이러한 군집 알고리즘을 발표했습니다. 그는 새(bird)와 발음이 비슷하면서 '새와 같은 물체(bird-like object)'를 지칭하는 보이드(boid)라는 용어를 만들어 냈습니다. 그는 각 유닛에 적용할 세 가지 간단한 규칙을 제안했습니다.

- **분리(Separation):** 각 보이드는 이웃한 보이드와 부딪히는 것을 피하기 위해 최소한의 거리를 유지해야 합니다(단거리 척력, short-range repulsion).
- **정렬(Alignment):** 각 보이드는 이웃 보이드의 평균 방향에 자신을 맞추고 플록(flock, 무리)으로서 같은 속도로 이동해야 합니다.
- **응집(Cohesion):** 각 보이드는 그룹의 질량 중심(center of mass)으로 이끌립니다(장거리 인력, long-range attraction).

새들의 사실적이고 적절히 복잡한 군집 행동(flocking behavior)을 구현하는 데 이 세 가지 간단한 규칙이면 충분합니다. 이는 새에게만 적용되는 것은 아닙니다. 군집 행동은 군중(crowd)을 모델링하거나, 심지어 게임 중 플레이어를 따라다니는 몇 명의 NPC를 모델링하는 데도 유용합니다.

우리는 5장 '플로킹'에서 유니티에 이러한 플로킹 시스템을 구현하는 방법을 배울 것입니다.

경로 추종 및 스티어링 (Path following and steering)

때로는 AI 캐릭터가 게임 월드를 돌아다니며 대략적으로 안내되거나 혹은 완전히 정의된 경로를 따라가게(follow a path) 하고 싶을 때가 있습니다. 예를 들어 레이싱 게임에서 AI 상대는 도로를 탐색(navigate)해야 합니다. 이러한 경우 우리의 플로킹 보이드 알고리즘과 같은 단순한 반응형 알고리즘(reactive algorithms)만으로는 이 문제를 해결하기에 충분하지 않습니다. 결국 이 모든 것은 실제 움직임과 스티어링 동작(steering behaviors, 조향 동작)을 다루는 것으로 귀결됩니다. AI 캐릭터의 스티어링 동작은 지난 수십 년 동안 지속되어 온 연구 주제입니다.

이 분야의 주목할 만한 논문은 다시 한 번 크레이그 레이놀즈(Craig Reynolds)가 1999년 게임 개발자 콘퍼런스(GDC)에서 발표한 Steering Behaviors for Autonomous Characters입니다. 그는 스티어링 동작을 다음의 세 계층(layers)으로 분류했습니다:

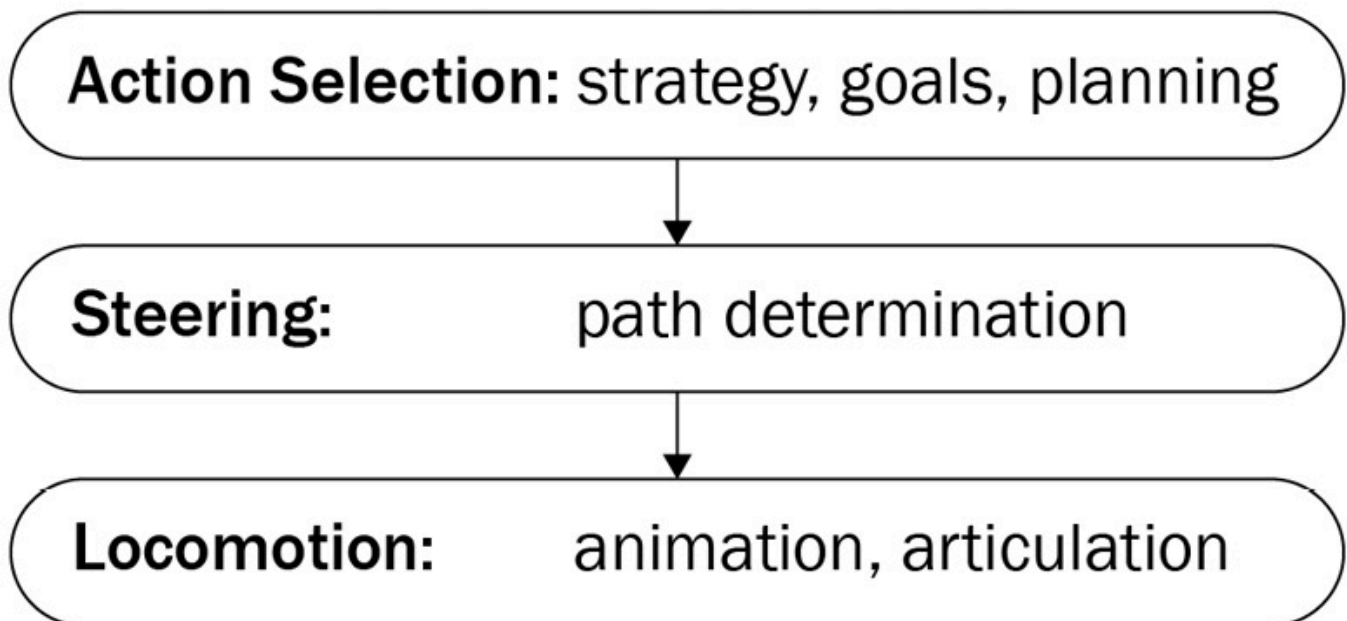


그림 1.2 – 동작 행동의 계층 구조

이 계층들을 이해하기 위해 한 가지 예를 들어보겠습니다. 어느 더운 여름날 오후, 책상에서 일을 하고 있다고 상상해 보십시오. 목이 마른 당신은 차가운 아이스티 한 잔이 마시고 싶습니다. 여기서 첫 번째 계층에서 시작합니다. 차가운 아이스티 한 잔을 원하며(목표 설정), 그것을 얻기 위해 무엇을 해야 할지 계획합니다. 아마도 부엌으로 이동해서(책상 밑에 미니 냉장고가 없다면) 빈 잔을 가져온 다음, 냉장고로 가서 문을 열고 아이스티를 꺼내야 할 것입니다(높은 수준의 계획을 수립함).

이제 두 번째 계층으로 이동합니다. 책상에서 부엌까지 직진으로 뚫려 있는 길이 아니라면, 경로를 결정해야 합니다. 책상을 돌아, 복도를 지나, 컵이 있는 찬장에 도달할 때까지 주방 가구를 피해 이동하는 등의 과정을 거칩니다. 이제 경로가 생겼으니, 세 번째 계층인 경로 따라 걷기(walking the path)로 넘어갈 차례입니다. 이 예에서 세 번째 계층은 여러분의 신체, 골격, 근육이 경로를 따라 움직이도록 기능하는 것으로 나타납니다.

정보

걱정하지 마세요. 이 세 가지 계층을 모두 마스터할 필요는 없습니다. AI 프로그래머로서 여러분은 처음 두 개의 계층에만 집중하면 됩니다. 세 번째 계층은 일반적으로 그래픽 프로그래머, 특히 애니메이터가 처리합니다.

이 세 가지 계층을 설명한 후, 크레이그 레이놀즈는 개별 AI 캐릭터를 위한 표준 스티어링 동작을 설계하고 구현하는 방법을 설명합니다. 이러한 행동에는 탐색과 도피(seek and flee), 추적과 회피(pursue and evade), 배회(wander), 도착(arrival), 장애물 회피(obstacle avoidance), 벽 따라가기(wall following) 및 경로 추종(path following)이 포함됩니다.

우리는 6장 '경로 추종 및 스티어링 동작'에서 이러한 행동 중 일부를 유니티로 구현해 볼 것입니다.

A 경로 탐색(A pathfinding)

몬스터나 적들이 장애물을 피하면서 플레이어를 따라다니거나 특정 지점으로 이동하는 게임을 많이 볼 수 있습니다. 전형적인 RTS 게임을 예로 들어 보겠습니다. 유닛 그룹을 선택하고 이동시킬 위치를 클릭하거나 적 유닛을 클릭하여 공격하게 할 수 있습니다.

그러면 유닛들은 장애물과 충돌하지 않고 목표에 도달할 방법을 찾아야 합니다. 물론 적 유닛들도 똑같이 할 수 있어야 합니다. 장벽(barriers)은 유닛에 따라 다를 수 있습니다. 예를 들어 공군 유닛은 산을 넘어갈 수 있지만, 지상이나 포병 유닛은 산을 우회할 길을 찾아야 합니다.

A*(A-스타라고 발음함)는 뛰어난 성능, 정확성, 구현 용이성 덕분에 게임에서 널리 사용되는 경로 탐색 알고리즘입니다. 예시를 통해 작동 방식을 살펴보겠습니다. 유닛이 A 지점에서 B 지점으로 이동하길 원하지만 중간에 벽이 있어 타길을 향해 직선으로 갈 수 없다고 해보겠습니다. 따라서 벽을 피하면서 B 지점까지 갈 길을 찾아야 합니다.



그림 1.3 – 맵의 탑다운 뷰(Top-down view)

이것은 간단한 2D 예제지만, 3D 환경에도 같은 아이디어를 적용할 수 있습니다. A 지점에서 B 지점까지의 경로를 찾으려면 장애물의 위치와 같은 맵에 대한 더 많은 정보가 필요합니다. 이를 위해 전체 맵을 다음 다이어그램과 같이 격자(grid) 형식으로 나타내는 작은 타일들로 분할할 수 있습니다.

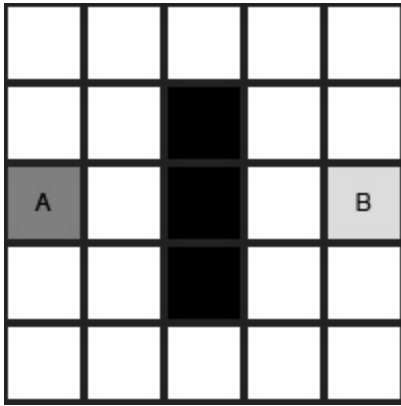


그림 1.4 – 2D 그리드로 표현된 맵

타일은 육각형(hexagons)이나 삼각형과 같은 다른 모양일 수도 있습니다. 각 모양에는 장점이 있습니다. 예를 들어, 육각형 타일은 대각선 이동의 문제(목표 육각형을 둘러싼 모든 육각형이 동일한 거리에 있음)가 없기 때문에 편리합니다. 하지만 이 예제에서는 그리드에 대해 생각할 때 가장 먼저 떠오르는 직관적인 형태인 사각형 타일을 사용했습니다.

이제 이 맵을 작은 2D 배열(2D array)로 참조할 수 있습니다.

우리는 맵을 총 25개의 타일로 구성된 5x5 사각형 타일 그리드로 표현할 수 있습니다. 이제 목표에 도달하기 위한 최적의 경로 탐색을 시작할 수 있습니다. 어떻게 해야 할까요? 장애물이 없고 시작 타일과 인접해 있는 각 타일의 이동 점수를 계산한 다음, 가장 비용이 낮은 타일을 선택하는 방식으로 진행합니다.

대각선 이동을 고려하지 않는다면, 플레이어에게 인접한 타일은 4개가 가능합니다. 이제 각 타일에 대한 이동 점수를 계산하기 위해 두 개의 숫자를 사용해야 합니다. 이것들을 G와 H라고 부르겠습니다. 여기서 G는 시작 타일에서 현재 타일로 이동하는 데 드는 비용이고, H는 현재 타일에서 타겟 타일에 도달하기 위한 예상 비용입니다.

G와 H의 합을 $F(F = G + H)$ 라고 부르겠습니다. 즉, 해당 타일의 최종 점수입니다.

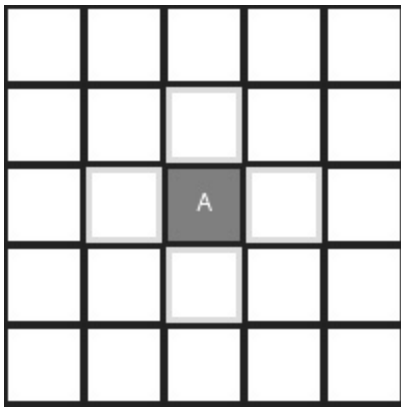


그림 1.5 – 유효한 인접 타일

이 예제에서 H 를 추정하기 위해 우리는 맨해튼 거리(Manhattan length, 또는 택시 기하학(taxicab geometry)으로도 알려짐)라는 간단한 방법을 사용할 것입니다. 이 방법에 따르면 A와 B 사이의 거리(비용)는 A와 B 사이의 수평 타일 수와 수직 타일 수를 더한 값입니다.

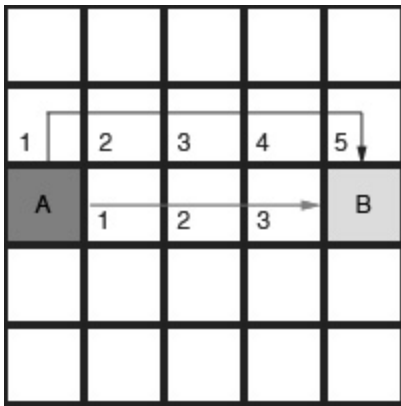


그림 1.6 – G 계산

반면 G 값은 탐색 중에 지금까지 소요된 비용을 나타냅니다. 이전 다이어그램은 두 가지 다른 경로에 대한 G 의 계산을 보여줍니다. 현재 G 를 계산하려면 이전 타일의 G 점수에 1(타일 하나를 이동하는 비용)을 더해야 합니다. 하지만 타일마다 다른 비용을 부여할 수도 있습니다. 예를 들어, (만약 고려한다면) 대각선 이동에 대해 더 높은 이동 비용을 설정하거나 연못이나 진흙탕으로 덮인 타일에 비용을 높게 줄 수 있습니다.

이제 G 를 구하는 방법을 알았으니 H 를 계산하는 방법을 알아보겠습니다. 다음 다이어그램은 다른 시작 타일들에 대한 H 값을 보여줍니다. 이 경우에도 맨해튼 거리를 사용합니다.

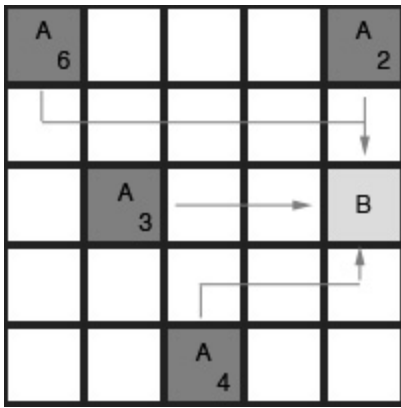


그림 1.7 – H 계산

G와 H를 구하는 방법을 알았으니, A에서 B까지의 최단 경로를 파악하기 위해 원래 예제로 돌아가 보겠습니다. 먼저 시작 타일을 선택하고 다음 다이어그램에 나타난 대로 모든 인접 타일을 수집해야 합니다. 그런 다음 각 타일의 하단 왼쪽과 오른쪽에 표시된 대로 G와 H 점수를 계산합니다. 마지막으로 G와 H를 더하여 최종 점수인 F를 얻습니다. 타일의 왼쪽 상단 모서리에서 F 점수를 볼 수 있습니다.

이제 F 점수가 가장 낮은 타일을 다음 타일로 선택하고, 이전 타일을 그 부모(parent)로 저장해야 합니다. 각 타일의 부모를 기록해 두는 것은 매우 중요한데, 나중에 이 역방향 링크(backlink)를 사용하여 끝에서부터 시작점까지 노드 순서를 역추적하여 최종 경로를 얻을 것이기 때문입니다. 이 예에서는 시작 위치의 오른쪽에 있는 타일을 선택하여 현재 타일로 간주해야 합니다.

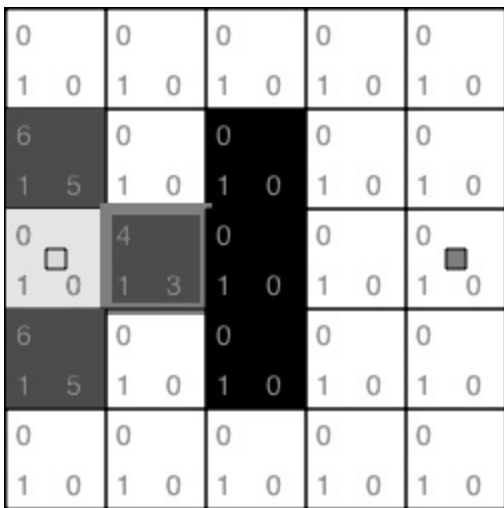


그림 1.8 – 시작 위치

현재 타일에서부터 유효한 인접 타일을 수집하는 과정을 반복합니다. 이번에는 비어있는 인접 타일이 위쪽 타일과 아래쪽 타일 단 2개뿐입니다(사실 왼쪽 타일은 이미 검사한 시작 타일이고 오른쪽 타일은 장애물이 차지하고 있습니다). 이 새로운 인접 타일들의 G와 H, 그리고 F 점수를 계산합니다.

이번에는 우리 맵 상에 점수가 6점으로 모두 동일한 네 개의 타일이 생겼습니다. 따라서 그중 아무 타일이나 선택할 수 있습니다. 사실, 어느 타일을 먼저 탐색하든 결국 최단 경로를 찾게 됩니다(이 진술 이면의 수학적 증명은 이 책의 범위를 벗어납니다).

0	0	0	0	0
1 0	1 0	1 0	1 0	1 0
6	6	0	0	0
1 5	2 4	1 0	1 0	1 0
0	4	0	0	0
1 0	1 3	1 0	1 0	1 0
6	6	0	0	0
1 5	2 4	1 0	1 0	1 0
0	0	0	0	0
1 0	1 0	1 0	1 0	1 0

그림 1.9 – 두 번째 단계

이 예제에서는 비용이 6인 타일 그룹 중에서 왼쪽 상단의 타일을 시작 위치로 선택했습니다. 다시 인접 타일을 검사해야 합니다. 이 단계에서는 계산된 F 점수가 8인 새로운 인접 타일이 하나만 있습니다. 현재로서는 가장 낮은 점수가 여전히 6이므로 점수가 6인 타일 중 아무거나 선택할 수 있습니다.

8	0	0	0	0
2 6	1 0	1 0	1 0	1 0
6	6	0	0	0
1 5	2 4	1 0	1 0	1 0
0	4	0	0	0
1 0	1 3	1 0	1 0	1 0
6	6	0	0	0
1 5	2 4	1 0	1 0	1 0
0	0	0	0	0
1 0	1 0	1 0	1 0	1 0

그림 1.10 – 세 번째 단계

목표 타일에 도달할 때까지 이 과정을 반복하면 비어있는 각 타일에 대한 모든 점수가 표시된 보드를 얻게 됩니다.

8	8	8	8	8
2 6	3 5	4 4	5 3	6 2
6	6	0	8	8
1 5	2 4	1 0	6 2	7 1
0	4	0	8	8
1 0	1 3	1 0	7 1	8 0
6	6	0	8	8
1 5	2 4	1 0	6 2	7 1
8	8	8	8	8
2 6	3 5	4 4	5 3	6 2

그림 1.11 – 목표 달성

이제 단 하나의 단계만 남았습니다. 각 노드에 저장했던 부모 링크(parent links)를 기억하시나요? 이제 타깃 타일에서 시작하여 저장된 부모 타일을 사용해 타일 목록을 역추적(trace back)해야 합니다. 결과 목록은 다음과 같은 형태의 경로가 될 것입니다.:

8	8	8	8	8
2 6	3 5	4 4	5 3	6 2
6	6	0	8	8
1 5	2 4	1 0	6 2	7 1
0	4	0	8	8
1 0	1 3	1 0	7 1	8 0
6	6	0	8	8
1 5	2 4	1 0	6 2	7 1
8	8	8	8	8
2 6	3 5	4 4	5 3	6 2

그림 1.12 – 역추적된 경로

여기서 설명한 것이 모든 경로 탐색 알고리즘의 기초이자 핵심인 A 경로 탐색 알고리즘입니다. 다행히 유니티 3.5부터 도입된 자동 내비게이션 메시(Navigation Mesh) 생성 및 내비메시 에이전트(NavMesh Agent)와 같은 몇 가지 새로운 기능 덕분에 게임에서 경로 탐색을 훨씬 쉽게 구현할 수 있습니다. 결과적으로, AI 캐릭터에 경로 탐색을 구현하기 위해 A에 대해 아무것도 알 필요가 없을 수도 있습니다. 그럼에도 불구하고 막후에서 시스템이 어떻게 작동하는지 아는 것은 탄탄한 AI 프로그래머가 되기 위해 필수적입니다.

다음 섹션에서 NavMesh에 대해 이야기하고, 8장 '내비게이션 메시'에서 더 자세히 다루겠습니다.

내비게이션 메시(Navigation meshes)

이제 A 경로 탐색 알고리즘의 기본을 알았으니, A에서 그리드를 사용하면 시작점과 목표 위치 사이의 최단 경로를 얻기 위해 많은 단계가 필요하다는 것을 알아챘을 것입니다. 별것 아닌 것처럼 보일 수 있지만, 텅 빈 수천 개의 타일이 있는 거대한 맵에서 타일 단위로 경로를 찾는 것은 컴퓨터 자원의 엄청난 낭비입니다. 따라서 게임에서는 컴퓨팅 자원을 적게 사용하는 간단하고 효과적인 방법으로 AI 캐릭터 이동의 가이드라인 역할을 하는 웨이포인트(waypoints)를 자주 사용합니다.

다음 다이어그램과 같이 AI 캐릭터를 A 지점에서 B 지점으로 이동시키고 싶고 3개의 웨이포인트를 설정했다고 가정해 보겠습니다.

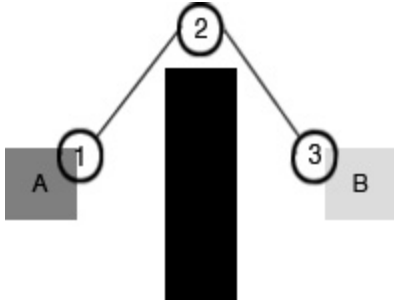


그림 1.13 – 웨이포인트(경유지)

이제 웨이포인트(전체 타일 수에 비하면 훨씬 적습니다)에 A* 알고리즘을 적용한 다음, 단순히 캐릭터를 웨이포인트에서 웨이포인트로 직선 이동시키기만 하면 됩니다.

하지만 웨이포인트에 문제가 없는 것은 아닙니다. 맵의 장애물을 업데이트하려면 어떻게 해야 할까요? 다음 다이어그램에서 보듯 업데이트된 맵에 맞게 웨이포인트를 다시 배치해야 할 것입니다.

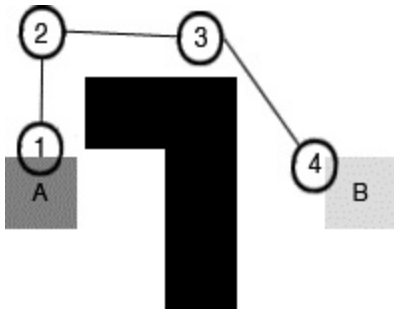


그림 1.14 – 새로운 웨이포인트

또한 각 노드를 따라 목표로 이동하면 캐릭터의 모습이 비현실적으로 보입니다. 예를 들어, 직선으로 이동하다가 방향을 급격하게 바꾸는 모습은 마치 테마파크 어트랙션의 기계 인형과 비슷합니다. 아니면 두 웨이포인트를 연결하는 경로가 장애물과 너무 가까울 수도 있습니다. 이전 다이어그램을 보십시오. 경로가 벽에 너무 가까운 곳에서는 AI 캐릭터가 벽과 충돌할 가능성이 큼니다.

그런 일이 발생하면 우리 AI는 다음 목표에 도달하기 위해 벽을 통과하려고 계속 시도하겠지만 불가능할 것이며 결국 거기에 갇히게 될 것입니다. 물론 스플라인(splines)을 사용하여 지그재그 경로를 부드럽게 만들거나 장애물 가장자리를 스치지 않도록 각 경로를 수동으로 확인하여 경로를 좀 더 사실적으로 만들 수는 있습니다. 그러나 문제는 웨이포인트에 두 노드를 연결하는 꺾은 선 이외의 환경 정보가 전혀 포함되어 있지 않다는 것입니다.

이러한 상황을 해결하려면 엄청나게 많은 수의 웨이포인트가 필요한데, 이는 관리하기가 매우 어렵습니다. 따라서 아주 단순한 게임을 제외하고는 그리드의 계산 비용(computational cost)을 수백 개의 웨이포인트를 관리하는 정신적, 설계적 비용(mental and design cost)과 교환해야 합니다.

다행히 더 나은 솔루션이 있습니다. 바로 내비게이션 메시(navigation mesh)를 사용하는 것입니다. 내비메시(NavMesh)라고도 불리는 내비게이션 메시는 사각형 기반 타일 그리드나 웨이포인트 그래프와 마찬가지로 우리 월드를 표현하는 데 사용할 수 있는 또 다른 형태의 그래프 구조입니다.

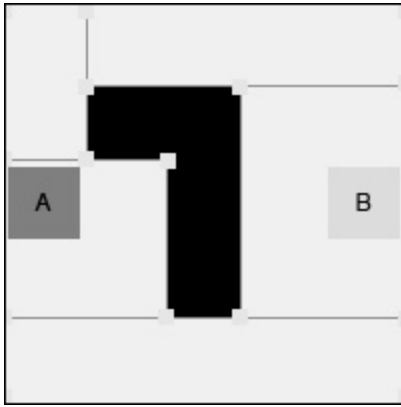


그림 1.15 – 내비게이션 메시

내비메시는 볼록 다각형(convex polygons)을 사용하여 AI 엔티티가 이동할 수 있는 맵 상의 영역을 나타냅니다. 내비메시 사용의 가장 중요한 이점은 웨이포인트 시스템보다 환경에 대한 훨씬 더 많은 정보를 포함하고 있다는 것입니다. 내비메시를 사용하면 AI 엔티티가 영역 내부를 자유롭게 이동할 수 있음을 알기 때문에 안전하게 경로를 자동으로 조정할 수 있습니다. 내비메시 사용의 또 다른 장점은 다양한 유형의 AI 엔티티에 대해 동일한 메시지를 사용할 수 있다는 것입니다. AI 엔티티마다 크기, 속도, 이동 능력 등 다양한 속성을 가질 수 있습니다. 예를 들어, 어떤 웨이포인트 세트는 인간 캐릭터에게 적합하지만, 날아다니는 생물이나 AI 제어 차량에는 잘 작동하지 않을 수 있습니다. 이들에게는 (이러한 요소가 초래하는 모든 문제를 수반하는) 다른 세트의 웨이포인트가 필요할지도 모릅니다.

하지만 씬(scene)을 기반으로 프로그래밍 방식으로 내비메시를 생성하는 것은 다소 복잡한 과정입니다. 다행히 유니티에는 내비메시 생성기가 내장되어 있습니다.

이 책은 핵심 AI 기법을 다루는 책이 아니므로 이러한 내비메시를 어떻게 생성하는지에 대해서는 깊이 들어가지 않을 것입니다. 대신 AI 캐릭터의 경로 탐색을 구현하기 위해 유니티의 내비메시를 효율적으로 사용하는 방법을 배울 것입니다.

행동 트리(Behavior trees)

행동 트리(Behavior trees)는 AI 캐릭터 결정의 기저에 있는 논리를 표현하고 제어하는 데 사용되는 또 다른 기법입니다. 헤일로(Halo)나 스포어(Spore) 같은 AAA 게임에서 활용되며 유명해졌습니다. 앞서 이 장에서 다루었던 FSM(유한 상태 기계)은 게임 이벤트에 대한 반응으로 여러 상태 간의 전이를 기반으로 AI 캐릭터의 논리를 정의하는 매우 직관적인 방법이었습니다. 하지만 FSM에는 확장하기(scale) 어렵고 재사용하기(reuse) 어렵다는 두 가지 주요 문제가 있습니다.

우리 캐릭터가 취하길 원하는 모든 시나리오를 지원하려면 많은 상태를 추가하고 무수히 많은 전이를 하 드코딩해야 합니다. 따라서 더 방대한 문제에 더 잘 확장되는 무언가가 필요합니다. 행동 트리는 올바른 방향으로 나아가는 합리적인 단계를 제시합니다.

이름에서 알 수 있듯이 행동 트리의 핵심은 트리 구조의 데이터 형태입니다. 이러한 트리의 잎사귀 (leaves)를 작업(tasks)이라고 하며, 캐릭터의 행동(예: 공격, 추적, 정찰, 숨기 등) 또는 감각 입력(예: 플 레이어가 근처에 있는가? 혹은 공격할 수 있을 만큼 가까운가?)을 나타냅니다. 반면 트리의 내부 노드 (internal nodes)는 제어 흐름 노드(control flow nodes)로 나타내며 트리의 실행을 안내합니다. 시퀀스 (Sequence, 순차), 셀렉터(Selector, 선택), 병렬(Parallel), 데코레이터(Decorator, 장식자) 등이 흔히 사용되는 제어 흐름 노드입니다.

이제 유한 상태 기계 섹션의 예제를 행동 트리를 사용하여 다시 구현해 봅시다. 먼저 모든 전이와 상태를 기본 작업으로 나눌 수 있습니다:

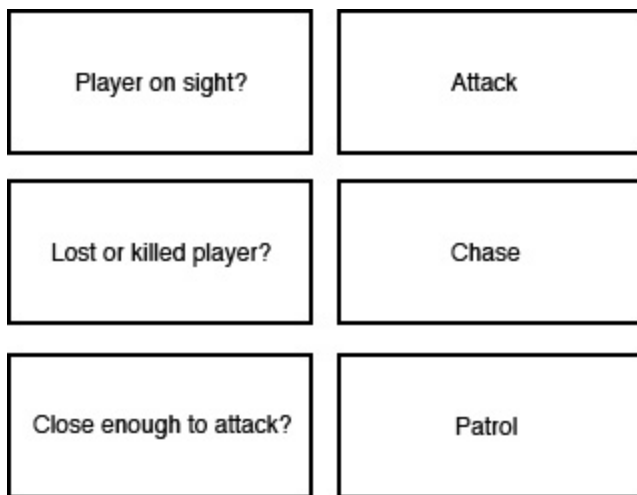


그림 1.16 – 작업(Tasks)

이제 셀렉터(Selector) 노드를 살펴보겠습니다. 우리는 셀렉터를 내부에 물음표가 있는 원으로 표시합니 다. 실행 시 셀렉터 노드는 성공(Success)으로 반환되는 첫 번째 작업/서브트리가 나올 때까지 모든 자 식 작업/서브트리를 순차적으로 실행하려고 시도합니다. 다시 말해 A, B, C, D라는 4개의 자식을 가진 셀렉터가 있다면, 셀렉터 노드는 먼저 A를 실행합니다. A가 실패(Failure)하면 셀렉터는 B를 실행합니 다. B가 실패하면 C를 실행하는 식입니다. 어떤 작업이라도 성공을 반환하면, 해당 작업이 완료되는 즉 시 셀렉터 노드는 성공을 반환합니다.

다음 예제에서 셀렉터 노드는 먼저 플레이어 공격을 선택합니다. 만약 '공격(Attack)' 작업이 성공을 반 환하면(즉, 플레이어가 공격 사거리 내에 있는 경우), 셀렉터 노드는 실행을 멈추고 부모 노드(있는 경우) 에게 성공을 반환합니다. 반면 공격 작업이 실패로 반환되면 셀렉터 노드는 '추적(Chase)' 작업으로 이 동합니다. 여기서 우리는 앞서 했던 것을 반복합니다. 추적 작업이 성공하면 셀렉터 노드도 성공합니다. 추적 작업이 실패하면 '정찰(Patrol)' 작업을 시도하는 식입니다.

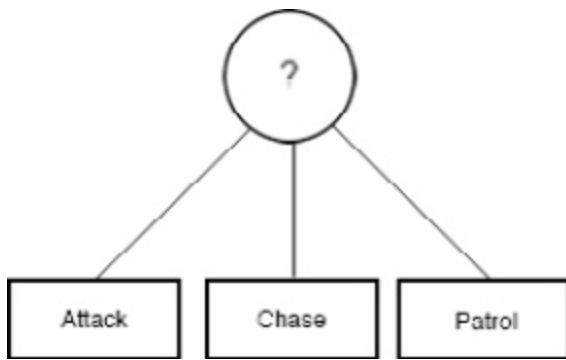


그림 1.17 – 선택기 노드

게임 상태를 확인하는 다른 종류의 작업들은 어떨까요? 우리는 이러한 작업들을 시퀀스(Sequence) 노드와 함께 사용하며, 시퀀스 노드는 주로 내부에 화살표가 있는 직사각형으로 표시됩니다. 시퀀스 노드는 셀렉터 노드와 유사하지만 아주 중요한 차이점이 있습니다. 바로 모든 서브트리들이 성공을 반환할 때만 성공 메시지를 반환한다는 것입니다. 다시 말해 A, B, C, D 4개의 자식을 가진 시퀀스가 있다면 시퀀스 노드는 A, B, C, D 순으로 실행할 것입니다. 모든 작업이 성공을 반환해야만 시퀀스가 성공을 반환합니다.

다음 예제에서 첫 번째 시퀀스 노드는 플레이어 캐릭터가 공격할 수 있을 만큼 가까이 있는지 확인합니다. 이 작업이 성공하면 다음 작업인 플레이어 공격으로 넘어갑니다. 공격 작업 역시 성공 메시지를 반환하면 전체 시퀀스는 성공으로 종료됩니다. 반면 '공격할 수 있을 만큼 가까운가?(Close Enough to Attack?)' 작업이 실패하면 시퀀스 노드는 공격 작업으로 진행하지 않고 부모 셀렉터 노드에 실패 상태를 반환합니다. 그러면 셀렉터는 시퀀스의 다음 작업인 '플레이어를 놓치거나 사살함(Lost or Killed Player)'을 선택하고 실행을 계속합니다.

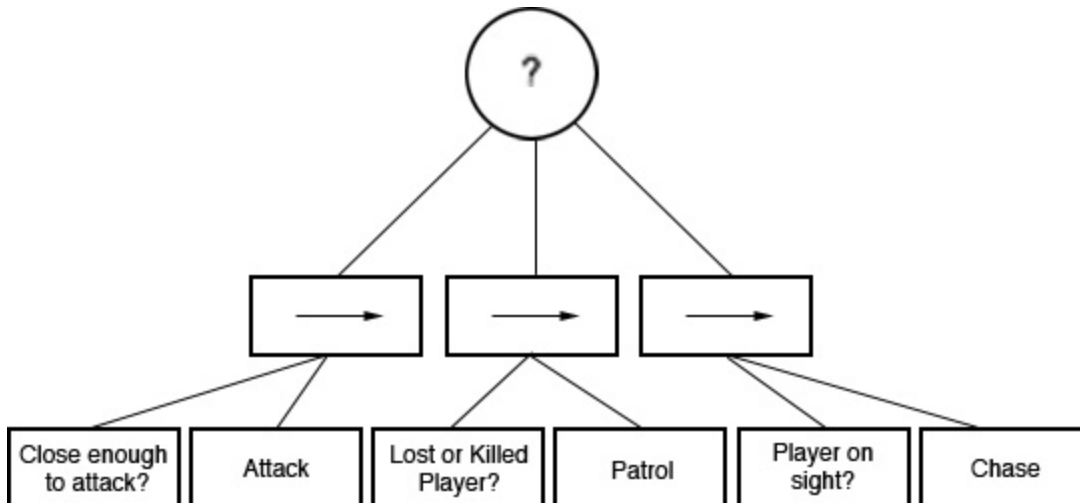


그림 1.18 – 시퀀스 작업 (작업 순서 지정)

다른 두 가지 공통 노드는 병렬(Parallel)과 데코레이터(Decorator)입니다. 시퀀스 및 셀렉터 노드가 자식 트리를 하나씩 순서대로 실행하는 반면, 병렬 노드는 모든 자식 작업을 동시에 실행합니다. 데코레이터는 자식을 하나만 갖는 또 다른 유형의 노드입니다. 단일 자식 서브트리의 동작을 변경하는 데 사용되며, 예를 들어 여러 번 실행하게 하거나 서브트리의 결과를 반전시킵니다(서브트리가 성공 메시지를 반환하면 데코레이터가 실패를 반환하고, 그 반대도 마찬가지입니다).

9장 '행동 트리'에서 유니티에 기본적인 행동 트리 시스템을 구현하는 방법을 배울 것입니다.

운동 /로코모션 (Locomotion)

동물(인간 포함)은 환경을 돌아다닐 수 있게 해주는 매우 복잡한 근골격계(musculoskeletal system)를 가지고 있습니다. 또한 이러한 시스템을 어떻게 사용할지 지시하는 정교한 뇌도 가지고 있습니다. 예를 들어, 우리는 사다리, 계단 또는 평탄하지 않은 지형을 오를 때 발을 어디에 디벼야 할지 본능적으로 알며, 취하고자 하는 화려한 포즈들을 안정화하기 위해 신체의 균형을 잡는 방법도 알고 있습니다. 우리는 뼈, 근육, 관절 및 기타 조직들, 다시 말해 총체적으로 우리의 이동 시스템(locomotor system)이라 일컬어지는 부분들을 제어하는 뇌를 사용하여 이 모든 것을 할 수 있습니다.

이제 이것을 게임 개발의 관점에 놓아 보겠습니다. 고르지 않은 표면이나 작은 경사로를 걸어야 하는 인간 캐릭터가 있고, 걷기 사이클에 대한 애니메이션이 단 하나뿐이라고 해보겠습니다. 가상 캐릭터에 이동 시스템이 없다면 다음과 같은 모습이 될 것입니다.

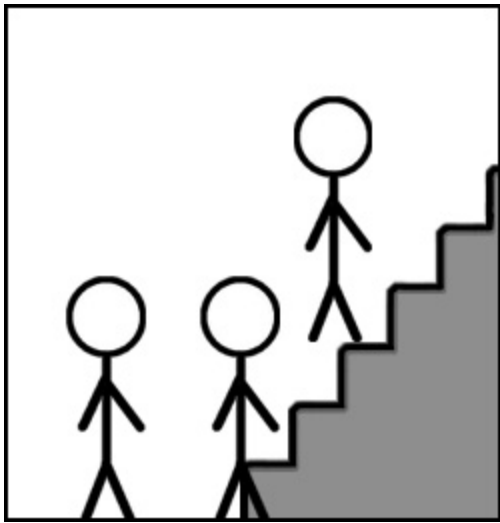


그림 1.19 – 이동(로코모션) 없이 계단 오르기

먼저 걷기 애니메이션을 재생하고 플레이어를 앞으로 이동시킵니다. 하지만 이제 캐릭터는 표면을 통과(관통)하고 있습니다. 따라서 충돌 감지 시스템(collision detection system)은 이 불가능한 구성을 막기 위해 캐릭터를 표면 위로 끌어올립니다.

이제 우리가 현실에서 계단을 어떻게 오르는지 살펴봅시다. 우리는 계단에 발을 단단히 딛고 힘을 사용하여 나머지 몸을 다음 계단으로 끌어올립니다. 하지만 게임에서 이러한 수준의 현실성을 구현하는 것은 간단하지 않습니다. 사다리 오르기, 계단 걷기/뛰기 등 다양한 시나리오에 맞는 수많은 애니메이션이 필요할 것입니다. 따라서 과거에는 애니메이터가 많은 대형 스튜디오에서만 이를 구현해낼 수 있었습니다. 하지만 오늘날에는 이를 위한 자동화된 시스템이 있습니다.



그림 1.20 – 역운동학(IK)을 위한 Unity 확장 기능

이 시스템은 우리의 애니메이션 된 걷기/달리기 사이클을 자동으로 블렌딩(blend)하고 플레이어 다리 뼈의 움직임을 조정하여 플레이어의 발이 땅을 올바르게 딛도록 보장할 수 있습니다(문헌에서는 이를 역운동학(inverse kinematics, IK)이라고 합니다). 또한 특정 속도와 방향에 맞춰 설계된 애니메이션을 계단이나 경사면 등 어떤 표면에서도 어떠한 속도와 방향으로든 조정할 수 있습니다. 6장 '경로 추종 및 스티어링 동작'에서는 이 로코모션 시스템을 사용하여 AI 캐릭터에 현실적인 움직임을 적용하는 방법을 배울 것입니다.

요약

이 장에서 우리는 게임 AI와 학술적 AI의 목표가 다르다는 것을 배웠습니다. 학계의 AI 연구자들은 복잡한 상황에서 인간을 대체하는 것을 궁극적인 목표로 삼아 현실 세계의 문제를 해결하고 인간의 지능과 경쟁하는 AI 알고리즘을 개발하려고 노력합니다. 반면, 게임 AI는 플레이어를 즐겁게 하려는 궁극적인 목표와 함께, 플레이어에게 똑똑해 보이도록 제한된 자원을 가진 NPC를 만드는 데 초점을 맞춥니다. 게임에서 AI의 목표는 게임을 더 재미있게 즐길 수 있도록 도전적인 상대를 제공하는 것입니다.

또한 FSM, 무작위성과 확률, 센서, 입력 시스템, 플로킹 및 군집 행동, 경로 추종 및 스티어링 동작, AI 경로 탐색, 내비게이션 메시 생성 및 행동 트리 등 게임에서 사용되는 다양한 AI 기법에 대해서도 배웠습니다.

우리는 다음 장들에서 유니티 엔진 내부에 이러한 기법을 구현하는 방법을 배울 것입니다. 다음 장에서는 가장 기본적인 내용인 FSM부터 시작하겠습니다.